**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Фронт-энд разработка

Отчет

# Лабораторная работа 2
# Взаимодействие с внешним API

Выполнил:

Чунихина Валерия

Группа J3210

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

**Задача**

Привязать то, что мы делали в ЛР1 к внешнему API средствами fetch/axios/xhr. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию, как в примерах, которые мы рассматривали в рамках тем "Иммитация работы с API".

Сайт: Социальная платформа для обмена моделями и датасетами (AI Model & Dataset Hub)

- Вход
- Регистрация
- Личный кабинет пользователя (загруженные модели/датасеты, подписки)
- Поиск моделей и датасетов с фильтрацией по задаче, лицензии, размеру и фреймворку
- Страница модели/датасета (описание, метрики, примеры использования, загрузки)
- Социальные функции (звезды, форки, обсуждения, процессы воспроизводимости)

Что добавлено в этой работе:

- Реализация получения данных из открытого Hugging Face API
- Имитация работы бэкенда с помощью json-server
- Реализация полноценной авторизации (вход и регистрация)
- Динамическое отображение данных карточек моделей и трендов
- Сетка для вывода данных (1 крупная карточка + ряды меньших карточек)

- Привязка данных пользователя, его личных моделей, к его профилю

## Ход работы

1. Я использовала json-server в качестве демонстрации бэкенда. В файле db.json есть две основные коллекции:
   - users: для хранения учетных данных пользователей (имя, email, пароль).
   - models: для хранения личных проектов пользователей, созданных через форму Создать сад. У модели в базе есть поле userId для реализации связи между пользователем и его контентом.

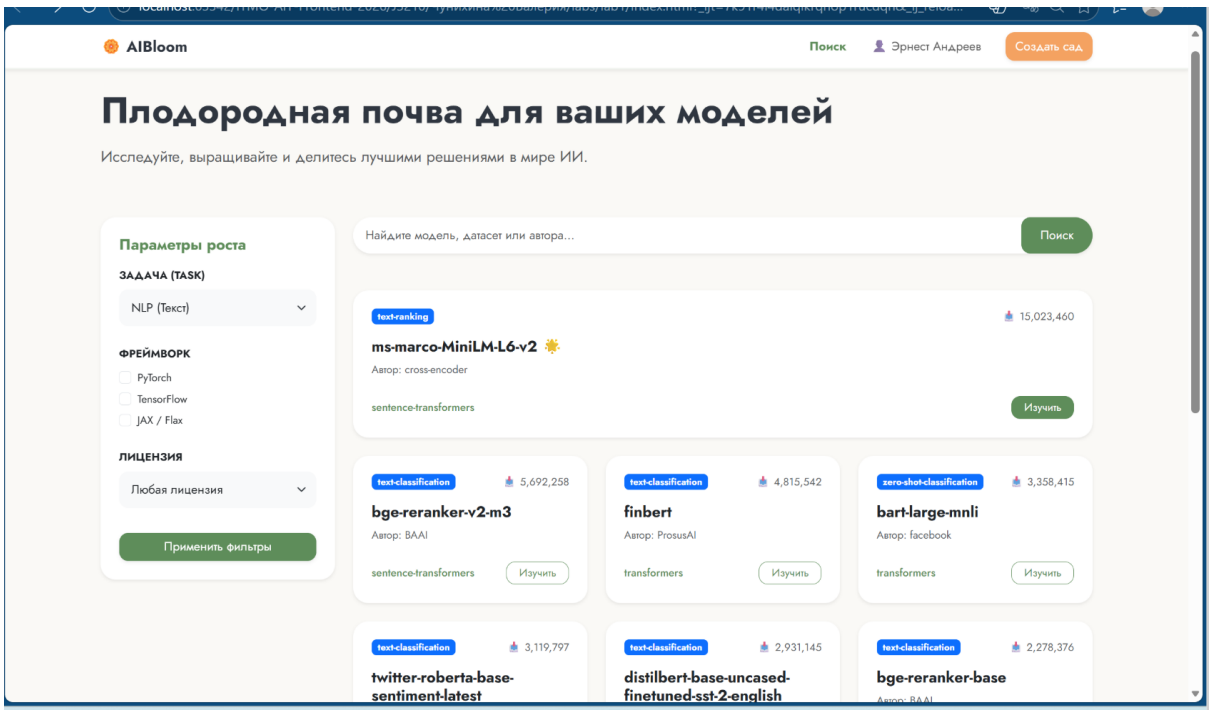

```
{
  "users": [
    {
      "name": "Валерия Чунихина",
      "email": "chunikhinaval@gmail.com",
      "password": "777lera",
      "id": "1"
    },
    {
      "name": "Саша Уракова",
      "email": "urakovasasha@gmail.com",
      "password": "6666sasha",
      "id": "7mVy544WgEw"
    },
```

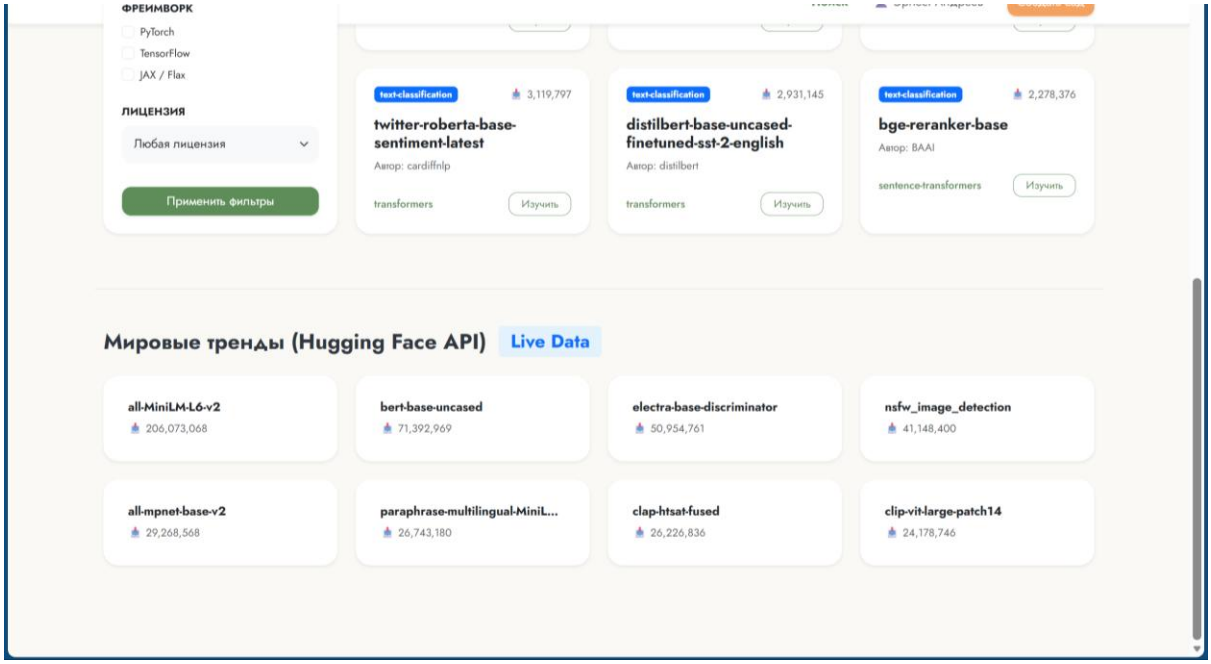
```
"models": [
  {
    "id": "1",
    "userId": "Kvh2BK6gcts",
    "name": "Forest-Vision v2",
    "type": "model",
    "task": "Computer Vision",
    "framework": "PyTorch",
    "license": "MIT",
    "size": "450MB",
    "stars": 1205,
    "description": "Анализ состояния лесов по спутниковым снимкам."
  },
  {
    "id": "2",
    "userId": "7mVy544WgEw",
    "name": "Russian-GPT-Flora",
    "type": "dataset",
    "task": "NLP",
    "framework": "Transformers",
    "license": "Apache 2.0",
    "size": "1.2GB",
    "stars": 850,
    "description": "Датасет описаний растений."
```

2. Я реализовала взаимодействие с внешним API для получения живых данных о нейросетях.
   Для этого в коде я использовала метод fetch для асинхронных запросов к huggingface.co/api/models.
   Реализовала обработку параметров сортировки по популярности и фильтрации.
   Реализовала аналогичным образом (без фильтрации) Тренды в нижней части главной страницы.

   Теперь здесь динамическая генерация контента
   Как это было реализовано:

- Реализация алгоритма сетки 1 + 3 + 3. Первая (самая популярная) модель выводится в крупную колонку col-12, а остальные в сетку col-md-4.
- Функция applyFilters собирает состояние чекбоксов (PyTorch, TensorFlow) и селектов (Task, License), формирует query string и отправляет запрос к API.
- Добавление индикаторов загрузки (Spinners) для ожидании ответа от сервера.

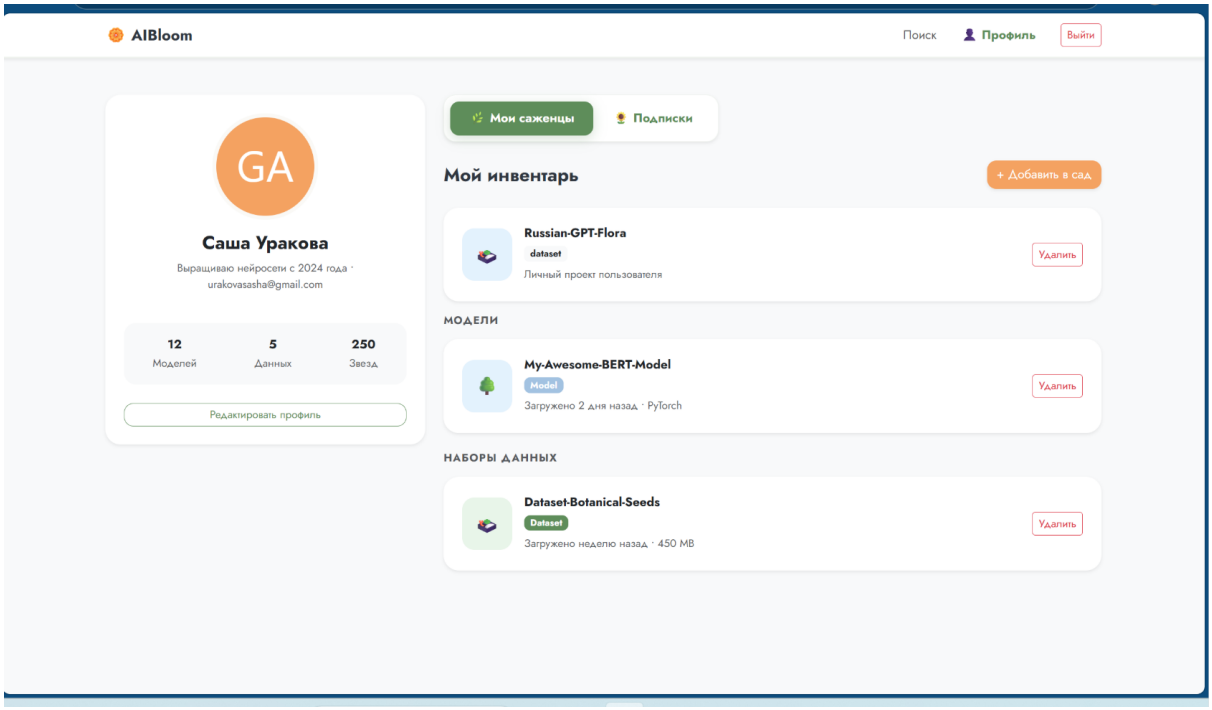





**3.** Вместо alert, я реализовала реальную проверку данных.

Логика:

- При регистрации метод post отправляет объект нового пользователя в db.json.
- При входе метод get с фильтрами по email и password проверяет наличие пользователя в базе.
- При успешном входе данные пользователя сохраняются в localStorage под ключом currentUser. Это позволяет сохранять состояние авторизации при переходе между страницами.




4. Получается у меня два источника данных: на главной странице отображаются общие тренды (API), а в профиле только те модели, которые принадлежат текущему пользователю.

   Как в коде:

- Запрос с фильтром: fetch('${API_URL}/models?userId=${currentUser.id').
- Реализовала возможность удаления своей модели из базы (метод delete) и добавления новой через модальное окно (метод post).
- Динамическая смена имени на имя авторизованного пользователя из базы.

5. Файл script.js был переработан для поддержки асинхронности (async/await).

   Добавлены:

- Обработка заголовков запросов (Headers: Content-Type: application/json).
- Использование URLSearchParams для удобной сборки параметров фильтрации.
- Логика выхода (Logout) через очистку localStorage.

**Вывод**

В ходе выполнения лабораторной работы мною были освоены принципы взаимодействия приложения с API.

Получилось работать с асинхронными запросами (fetch) и обрабатывать JSON-ответы. А также настраивать моковое API для тестирования бизнес-логики приложения без реального бэкенда.

Также организовала систему аутентификации на стороне клиента и связала данные между различными коллекциями базы данных. И научилась динамически изменять структуру DOM на основе данных, полученных извне.

В результате мой проект AIBloom превратился из статического макета в функциональное веб-приложение.